



THE CHINESE UNIVERSITY OF HONG KONG, SHENZHEN

CSC 3170

Report for NBA Database

120090710 Jiannan Xiao

117010435 Kim Sangyoon

120090319 Donghao Wu

119010343 Hong Xiao

Content

1. Introduction and motivation	3
2. Design	3
2.1 E-R Diagram	
2.2 Relational Schemas	
2.3 Entities and Attributes Identification	
2.4 Functional Dependencies and Normalized Form	
3. Implementation	4
3.1 Build the Database	
3.1.1 Server and Port	
3.1.2 Build the Database using Django	
3.1.3 Index	
3.2 Insert the Data	
3.3 Connect the Database	
3.3.1 URL	
3.3.2 Django	
3.3.3 Xadmin	
4. Result	6
5. Data Mining.....	7
5.1 Model (Predict the Team Rank)	
5.1.1 Simplified PageRank	
5.1.2 Modified PageRank	
5.1.3 PageRank in Sports	
5.1.4 Result	
5.2 Other Data Visualization and Analysis	
5.2.1 Scoring List of Players	
5.2.2 Hit Rate Distribution	
5.2.3 Average point distribution	
6. Conclusion and self-evaluation	12
7. References	12
8. Appendices	12

1. Introduction and motivation

The NBA is the best professional basketball league globally with 30 teams from the United States and Canada, the dream stage of every basketball player, and the American professional basketball association that runs the NBA league. To hold such a grand match, the organizer needs to process large amounts of data efficiently and correctly. Thus, we designed a database that helps the organizer store and operate the data easily and correctly, making the event can be orderly and smooth.

2. Design

2.1 E-R Diagram

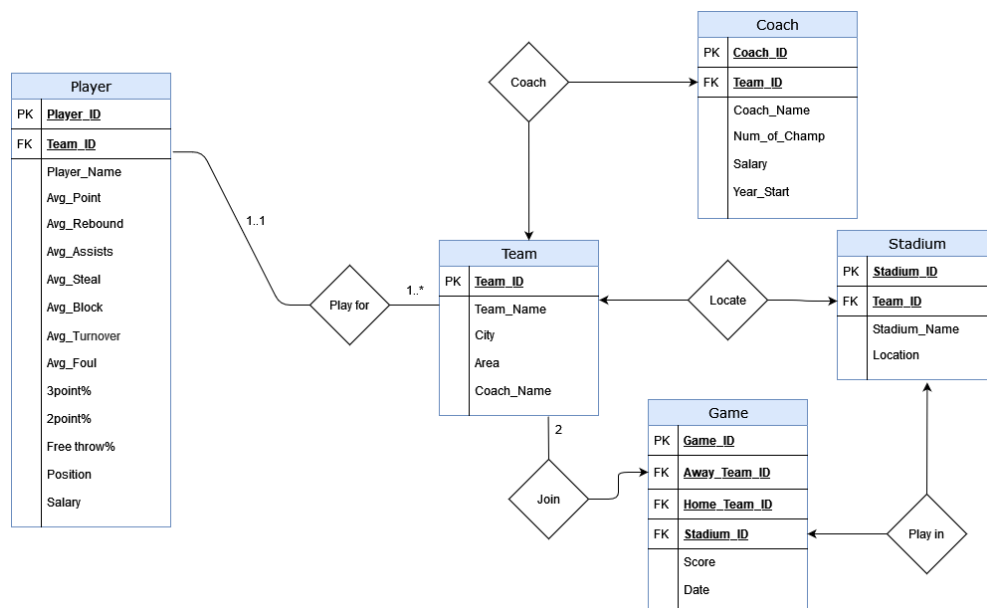


Figure 1. The E-R diagram of the database

2.2 Relational Schemas

Player(Player_ID, Team_ID, Player_Name, Avg_Point, Avg_Assists, Avg_Steal, Avg_Block, Avg_Turnover, Avg_Foul, 3point%, 2point%, Free throw%, Position, Salary)

Team_ID is the foreign key that references the Team table.

Team(Team_ID, Team_Name, City, Area, Coach_Name)

Coach(Coach_ID, Team_ID, Coach_Name, Num_of_Champ, Salary, Year_Start)

Team_ID is the foreign key that references the Team table.

Game(Game_ID, Away_Team_ID, Home_Team_ID, Stadium_ID, Score, Date)

Away_Team_ID is the foreign key that references the Team table.

Home_Team_ID is the foreign key that references the Team table.

Stadium_ID is the foreign key that references the Stadium table.

Stadium(Stadium_ID, Team_ID, Stadium_Name, Location)

Team_ID is the foreign key that references the Team table.

Play for(Player_ID, Team_ID)

Coach(Team_ID, Coach_ID)

Locate(Team_ID, Stadium_ID)

Play in(Stadium_ID, Game_ID)

Join(Team_ID, Game_ID)

2.3 Entities and Attributes Identification

Our database contains five tables. They are Player, Team, Coach, Game, and Stadium. The detailed information about them will be shown in the appendices 1.

2.4 Functional Dependencies and Normalized Form

Player_ID → Team_ID, Player_Name, Avg_Point, Avg_Assists, Avg_Steal,
Avg_Block, Avg_Turnover, Avg_Foul, 3point%, 2point%, Free_throw%,
Position, Salary

Team_ID → Team_Name, City, Area, Coach_Name

Coach_ID → Team_ID, Coach_Name, Num_of_Champ, Salary, Year_Start

Game_ID → Away_Team_ID, Home_Team_ID, Stadium_ID, Score, Date

Stadium_ID → Team_ID, Stadium_Name, Location

We can see from the above functional dependencies, that the left-hand sides are all primary keys of each table, which means the left-hand sides are superkeys of R. This satisfies the general definition of the third normal form. As a result, our database structure is in 3NF.

3. Implementation

We implement the database as a administrator website. The basic idea of implementation has three steps: (1) lend a server and build the database on it. (2) get the data from some NBA website and insert it to the database (3) connect the database with the web page. The models defined in the DB_models/models.py, The djangoStart is used to run the website. The templates are in the NBAadmin folder.

3.1 build the database

3.1.1 Server and Port

To make the website be able to be visited by everyone, we first lend a server from tencent cloud, and run our project on the port 12000 of the server. Then we open the port for others to visit our website.



Figure 3.1. The server we lend

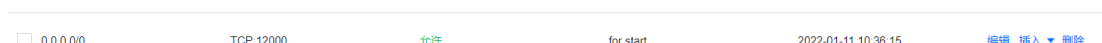


Figure 3.2. Open the port 12000

3.1.2 Build the Database using django

We use Django to build our database. We start an app called **DB_Models** and defined some models in the models.py under the app folder. It is similar with the create table in the SQL and it create some tables in the database.

3.1.3 Index

We use index to make searching more efficient. The following show the field of the models we use as indices;

- **Team:** Team_ID, Team_name
- **Player:** Player_ID, Player_name
- **Coach:** Coach_ID, Coach_name.
- **Player:** Team_ID, Player_name
- **Game:** Game_ID

We choose these attributes as indices because these attributes may been used frequently, and doing so can improve efficiency and save time. We set an attribute as index by setting **db_index= True** in Django.

3.2 Insert the Data

To populate the schemas with a reasonable amount of realistic data, we find the data in some website and got the .csv file. Then we use panda in python to handle these data and output a python file called **Insert.py** and use this file to insert data. To insert the data, we write sentence like (**player = models.Player(xxx=xxx) , player.save()**) to insert the data.

3.3 Connect the Database

This part we will show some tech we use briefly and show how we use them to build the web page and connect the database.

3.3.1 URL

Website directories and functions are stored as rules in url.py. It defined the rules of html and direct to the function that can handle the url.

3.3.2 Django

To connect the database with the web page, we use Django and Xadmin. If the request is sent, the data will in the form of HTML will be sent as well. To get the data, we use the POST method. We use **request.POST.get("Player_ID")** to get the data that sent by the form and the data has the name of the data in HTML is "id". To sent the data to HTML and display it, we use render. For example, use **return render(request, "xxx.html" , context)** can transfer the data to the webpage and display the result.

3.3.3 Xadmin

Xadmin is tool which can help build the UI. To use it we create a file called adminx.py in the DB_models folder, and connect the models and register it. For example, **xadmin.site.register(Team, TeamAdmin)** can register the Team models.

4. Result

The website address of our project is <http://1.15.247.42:12000/NBADBadmin/>. The username is **nan**, password is **csc3170nba**. The project may not be able to run on others computer, because some settings of the server is different (like IP Address and port).

The following data show how to add, search, delete the data.

Figure 4.1 add team

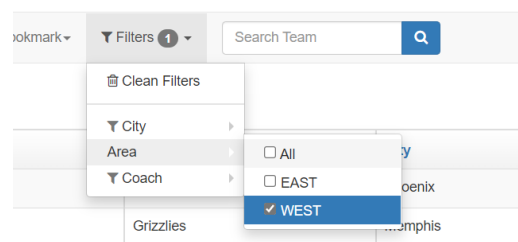


Figure 4.2 choose data(the team in WEST)

To add a team just click the add team button. Click the filter button can choose the data. Click the select button can delete the data.

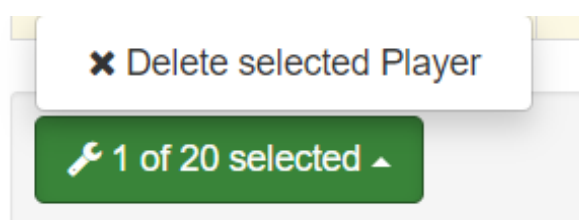


Figure 4.3 delete team

5. Data Mining

5.1 Model (Predict the Team Rank)

5.1.1 Simplified PageRank

PageRank was developed by Larry Page and Sergey Brin. It is based on the Markov process. The transition can be shown as:

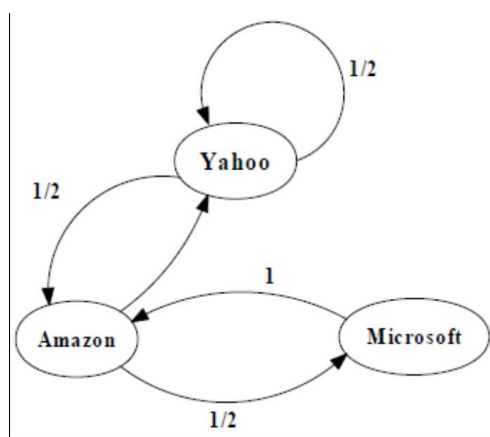
$$R(u) = c \sum_{v \in B_u} \frac{R(v)}{N_v}$$

which: u is a web page. B_u is the set of u 's backlinks. N_v denotes the number of forward links of page v . c is the normalization factor to make:

$$\|R\|_{L1} = 1, (\|R\|_{L1} = |R_1 + \dots + R_n|).$$

Each web page will link to another web pages with a weight, which means the probability transfer to the web page. The link relationship can be regarded as a directed graph. The graph can be expressed as a matrix.

Here is an example of Simplified PageRank:



For Yahoo and Amazon, they have two out-links, so both with probability $\frac{1}{2}$. For Microsoft, it only links to Amazon, so the probability is 1. Therefore the Matrix M is:

$$\begin{bmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & 1 \\ 0 & \frac{1}{2} & 0 \end{bmatrix}$$

There are two ways to calculate importance of web pages with the Matrix M: Power Method and Eigenvector Method.

1)Power Method:

Starting with the uniform distribution, the importance of each web page is $\frac{1}{3}$:

$$\pi_0 = \begin{bmatrix} Yahoo \\ Amazon \\ Microsoft \end{bmatrix} = \begin{bmatrix} \frac{1}{3} \\ \frac{1}{3} \\ \frac{1}{3} \end{bmatrix}$$

Calculate $\pi_{k+1} = Q\pi_k$ by iterations until π_k converges to R. In this example, we can calculate that:

$$R = \begin{bmatrix} \frac{2}{5} \\ \frac{2}{5} \\ \frac{1}{5} \end{bmatrix}$$

which is the final importance of these web pages.

2) Eigenvector Method:

Assuming R denotes the importance of web pages, R satisfies that $M \cdot R = R$. So equivalently, R is the eigenvector of M . So just calculate the eigenvector of M , it is the importance of web pages.

5.1.2 Modified PageRank

For some extreme case, some web page without any web page linked to itself will get 0 importance in the Simplified PageRank. For example, assuming that

$$M = \begin{bmatrix} 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & \frac{1}{2} & 1 \end{bmatrix}$$

calculate the eigenvector of M , it is $(0, 0, 1)$.

To avoid this problem, we bring the random walk idea to the Simplified PageRank. That is, for each web page, there is a probability of accessing any web page including itself. Then the transfer can be shown as:

$$R'(u) = c_1 \sum_{v \in B_u} \frac{R(v)}{N_v} + c_2 E(u)$$

which: $E(u)$ is the distribution of ranks of web pages that “users” jump to when they “gets bored” after successive links at random. c_1, c_2 are two nonnegative number satisfy $c_1 + c_2 = 1$. The Modified PageRank Model confirms that there will be no web page with importance 0 after several iterations.

5.1.3 PageRank in Sports

If we regard the difference score of one game between two teams as a directed edge, then the different score relationship can be seen as a directed graph, where the node is a team, and edge weight is the different score between the two teams.

let w_{ij} denotes when team j beats team i , the total different score. If team j never beats team i , $w_{ij} = 0$.

$$\text{let } w_{.j} = \sum_{k=1}^n w_{kj}, \quad g_{ij} = w_{ij} / w_{.j},$$

$$\text{then } \sum_{k=1}^n g_{kj} = 1,$$

n is the number of teams. g_{ij} can denote the proportion of different score that team j beats team i in the total difference score that team j beats other teams. If g_{ij} is large, that means team j wins a lot when j beat i . So if team i is a strong team, then it will contribute much to team j .

Let :

$$G = \begin{bmatrix} g_{11} & \cdots & g_{1n} \\ . & . & . \\ g_{n1} & \cdots & g_{nn} \end{bmatrix}$$

G can denote the different score relationship of the directed graph, it can be used to calculate the strength of each team. Also, to avoid that the strength of some team will converge to 0 after several iterations, we add the random walk idea to the model. The new model can be shown as:

$$Q = d * G + (1 - d) * E, \pi = Q\pi, \text{ where } d \in [0, 1], E = \begin{bmatrix} \frac{1}{n} & \cdots & \frac{1}{n} \\ . & . & . \\ \frac{1}{n} & \cdots & \frac{1}{n} \end{bmatrix}.$$

Our model choose $d=0.8$ and it gets a good result.

5.1.4 Result

The results of two methods are exactly same. Here are the teams that ranked by scores:

[BOS, PHO, GSW, MEM, UTA, MIA, TOR, MIN, ATL, DAL, MIL, CLE, PHI, DEN, CHO, NOP, LAC, BRK, CHI, NYK, WAS, POR, IND, SAS, LAL, OKC, SAC, ORL, DET, HOU]

And for the top four teams, they have all advanced the Quarter-Finals. That means our model has a good prediction on the team rank.

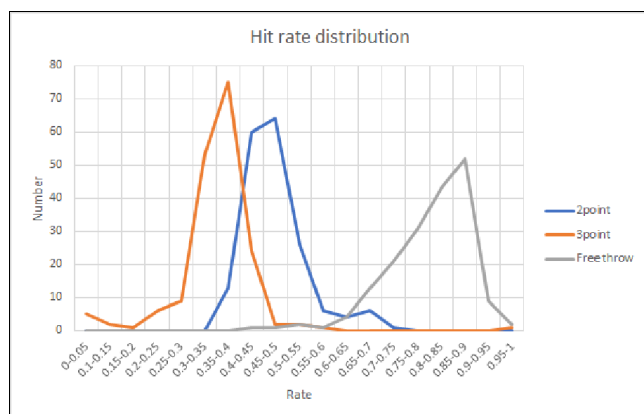
5.2 Other Data Visualization and Analysis

5.2.1 Scoring List of Players

Team	Player Name	Avg_Point
76ers	Joel Embiid	30.6
Lakers	Lebron James	30.3
Bucks	Giannis Antetokounmpo	29.9
Nets	Kevin Durant	29.9
Hawks	Trevor - Yang	28.4
Mavericks	Luka Doncic	28.4
Bulls	Demar derozan	27.9

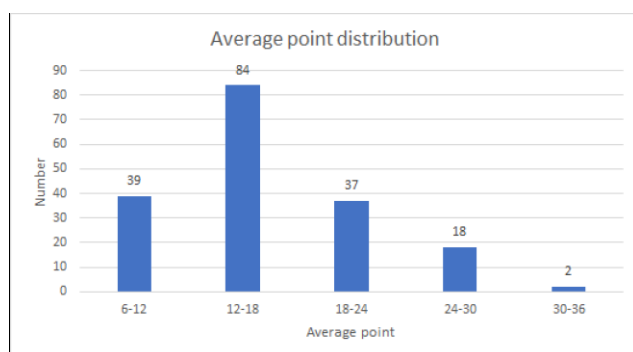
We show the top 7 players who have the highest scores. We can see that they are not in the same team. That is easy to understand, usually a high-scoring player needs a lot of shots. However, the total number of shots a team can take is limited, so it's difficult for multiple high-scoring players in a team.

5.2.2 Hit Rate Distribution



It shows the 2 point, 3 point and free throw hit distribution. We can see the 2 point hit rate concentrates on 0.45-0.55, where 3 point hit rate concentrates on 0.4-0.45 and free throw hit concentrates on 0.75-0.85.

5.2.3 Average point distribution



We choose the top 180 player with scores. Among them, most players have a score between 6 and 24. Only 20 players have scores more than 24.

6. Conclusion and self-evaluation

Our database successfully stores and processes the data of the NBA. We can easily find the information of each player, coach, team, game, and stadium, making the match can be held correctly and orderly. What's more, we can do some further analysis based on the data, such as predicting the winning team, analyzing the player's performance, and so on.

However, our database still has some defects. For example, we don't collect the previous coaching experiences of coaches. This might affect the result of our prediction.

7. References

Lazova, V. and Basnarkov, L. (2015). Pagerank approach to ranking national football teams. *arXiv preprint arXiv:1503.01331*.

Matthews, N. R., McClain, A., Smith, C. M., and Tennant, A. G. (2021). Application of pagerank algorithm to division i ncaa men's basketball as bracket formation and outcomepredictive utility. *Journal of Sports Analytics*, 7(1):1–9.

NBA China. (2015, march 23). *2021-2022 playoff data*. <https://m.china.nba.cn/statistics/>

NBA China. (2014, march 13). *NBA raw data download Advanced Sports Analytics*. <https://www.advancedsportsanalytics.com/nba-raw-data>

8. Appendices

A1. Entities and Attributes identification

Player: This is a table that contains the basic information of any player

- **Player_ID:** The player's identifier that determines other attributes. An integer of dynamic length.
- **Team_ID:** Describes the team that the player belongs to. It should be of the same format as that of the Team.Team_ID. Any player should register his team.
- **Player_Name:** The name of the player. A string with at most 35 characters.
- **Avg_Point:** The average score for the player of the entire tournament. A float number with at most 3 digits to the left of the decimal point, and 1 to the right of

it.

- **Avg_Rebound:** The average times for the player to do the rebound of the entire tournament. A float number with at most 3 to the left of the decimal point, and 1 to the right of it.
- **Avg_Assists:** The average times for the player to do the assist of the entire tournament. A float number with at most 3 to the left of the decimal point, and 1 to the right of it.
- **Avg_Steal:** The average times for the player to do the steal of the entire tournament. A float number with at most 3 to the left of the decimal point, and 1 to the right of it.
- **Avg_Block:** The average times for the player to do the block of the entire tournament. A float number with at most 3 to the left of the decimal point, and 1 to the right of it.
- **Avg_Turnover:** The average times for the player to do the turnover of the entire tournament. A float number with at most 3 to the left of the decimal point, and 1 to the right of it.
- **Avg_Foul:** The average times for the player fouls in the game of the entire tournament. A float number with at most 3 to the left of the decimal point, and 1 to the right of it.
- **3point%:** The rate for the player to successfully hit a three-point shot. A float number with at most 2 to the left of the decimal point, and 1 to the right of it.
- **2point%:** The rate for the player to successfully hit a two-point shot. A float number with at most 2 to the left of the decimal point, and 1 to the right of it.
- **Free throw%:** The rate for the player to successfully hit a free throw. A float number with at most 2 to the left of the decimal point, and 1 to the right of it.
- **Position:** The position of the player. A string with at most 15 characters.
- **Salary:** The real annual salary divided by 10000 dollars of the player. A float number with at most 4 to the left of the decimal point, and 2 to the right of it.

Team: This is a table that contains the basic information of any team

- **Team_ID:** The team's identifier that determines other attributes. An integer of dynamic length.
- **Team_Name:** The name of the team. A string with at most 20 characters.
- **City:** The city of the team. A string with at most 35 characters.
- **Area:** The east or west conference that the team belongs to. A string with at most 5 characters.
- **Coach_Name:** The name of the coach for the team. A string with at most 35 characters.

Coach: This is a table that contains the basic information of any coach

- **Coach_ID:** The coach's identifier that determines other attributes. An integer of dynamic length.
- **Team_ID:** Describes the team that the coach belongs to. It should be of the same format as that of the Team.Team_ID. Any coach should register his team.

- **Coach_Name:** The name of the coach. A string with at most 35 characters.
- **Num_of_Champ:** The number of championships won by the coach's team. An integer with at most 32 digits.
- **Salary:** The real annual salary divided by 10000 dollars of the coach. A float number with at most 4 to the left of the decimal point, and 2 to the right of it.
- **Year_Start:** The start year of the coach in this team. A date type with the format "mon dd yyyy".

Game: This is a table that contains the basic information of any game

- **Game_ID:** The game's identifier that determines other attributes. An integer of dynamic length.
- **Away_Team_ID:** Describes the away team id in this game. It should be of the same format as that of the Team.Team_ID.
- **Home_Team_ID:** Describes the home team id in this game. It should be of the same format as that of Team.Team_ID.
- **Stadium_ID:** Describes the stadium id in this game. It should be of the same format as that of Stadium.Stadium_ID.
- **Score:** The score of the away team versus the home team. A string with at most 10 characters.
- **Date:** The holding date of the game. A date type with the formate "mon dd yyyy".

Stadium: This is a table that contains the basic information of any stadium

- **Stadium_ID:** The stadium's identifier that determines other attributes. An integer of dynamic length.
- **Team_ID:** Describes the home team id in this stadium. It should be of the same format as that of Team.Team_ID.
- **Stadium_Name:** The name of the stadium. A string with at most 50 characters.
- **Location:** The city of the stadium. A string with at most 50 characters.